URBANFLOW: URBAN TRAFFIC GRAPH OPTIMIZATION ENGINE

A COURSE PROJECT REPORT

submitted by

AMAN ANTONY (MUT24CA013) **JENNIFER JOFFY (MUT24CA042)**

JOHN VARGHESE (MUT24CA044) **SANGEETHA SHALOM SAJI (MUT24CA065)**

as part of the course

PBCATS04 - ADVANCED GRAPH ALGORITHMS

to

The APJ Abdul Kalam Technological University in partial fulfilment of the requirements for the award of the Degree of

Bachelor of Technology

in

Computer Science and Engineering (Artificial Intelligence)

Department of Artificial Intelligence and Data Science

Muthoot Institute of Technology & Science

Kochi - Madurai - Tondi Pt Rd, Puthenkurish, Varikoli, Kochi, Kerala 682308

OCTOBER 2026

CERTIFICATE

This is to certify that the report entitled **"URBANFLOW: URBAN TRAFFIC GRAPH OPTIMIZATION ENGINE"**, submitted by **Aman Antony** (**MUT24CA013**), **Jennifer Joffy** (**MUT24CA042**), **John Varghese** (**MUT24CA044**), and **Sangeetha Shalom Saji** (**MUT24CA065**) to **Muthoot Institute of Technology & Science** for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence)**, is a bonafide record of the course project work carried out by them, as part of the course **PBCATS04 ADVANCED GRAPH ALGORITHMS**, under my supervision and guidance.

Place: Puthenkurish, Kochi
Date: 27/10/2026

Faculty-in-charge
Department of AI & DS

ABSTRACT

Rapid urbanisation and vehicular growth impose severe strains on road transportation systems, resulting in chronic traffic congestion, fuel waste, and elevated tailpipe carbon emissions. Traditional transportation interventions frequently evaluate road corridors in isolation, failing to anticipate non-linear network dynamics such as the Braess Paradox, where adding capacity or maintaining counter-productive shortcuts paradoxically elevates overall network travel latency. This project presents **UrbanFlow**, an advanced graph-theoretic simulation and optimization engine designed to evaluate, simulate, and resolve urban traffic congestion at metropolitan scale. Road networks are modeled as directed, weighted graphs $G = (V, E)$, incorporating real-world geospatial infrastructure extracted from OpenStreetMap (OSM) via OSMnx for Kochi, Kerala, comprising 490 junctions and 1,200 directed edges servicing 14 primary origin-destination (OD) corridors. Dynamic link traversal latencies are computed via the non-linear Bureau of Public Roads (BPR) function, and user-equilibrium traffic distributions are derived using the Method of Successive Averages (MSA) accelerated by Rust-backed Dijkstra routing. The system was bench-tested under both 100% baseline demand (27,300 vehicles per hour) and a 200% heavy congestion shock (54,600 vehicles per hour). Experimental results demonstrate that doubling demand induces an explosive 252.9% surge in average travel time (21.84 min to 77.07 min) and a $7.06\times$ rise in total network delay (9,935.2 to 70,137.7 vehicle-hours), expanding breakdown links ($V/C \geq 0.95$) from 112 to 295. UrbanFlow's combinatorial optimizer systematically discovered critical Braess Paradox shortcuts (notably on Road 1048), where converting bidirectional shortcuts into one-way corridors eliminates selfish detours, improving forward capacity by 40% and cutting citywide average travel latency by 20.35% under heavy congestion. Coordinated network interventions achieve a projected 30.69% latency reduction under 200% demand, saving over 21,500 vehicle-hours peak delay.

CO – PO / PSO ARTICULATION MATRIX

CO1: Explain fundamental graph concepts and algorithms to analyse and solve problems involving paths, cycles, trees, and network optimization. (K3)
CO2: Illustrate matching and covering problems in graphs using algorithms and theorems, including Hall's Condition and Hungarian Algorithm. (K3)
CO3: Apply connectivity and paths including Network Flow Ford-Fulkerson Labeling Algorithm, Max-Flow Min-Cut Theorem, and Menger's Proof. (K3)
CO4: Illustrate Vertex Coloring and Upper Bounds, Brooks' Theorem and Color-Critical Graphs, Counting Proper Colorings. (K3)
CO5: Apply concepts of planar graphs, Kuratowski's and Wagner's theorems, graph planarity, edge-coloring, TSP, and dominating sets. (K3)

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2
CO1	3	3	3	3	3	–	–	–	–	–	–	2	3	2
CO2	3	3	3	–	3	–	–	–	–	–	–	2	3	2
CO3	3	3	3	3	3	–	–	–	–	–	–	2	3	2
CO4	3	3	3	3	2	–	–	–	–	–	–	2	3	2
CO5	3	3	3	–	3	–	–	–	–	–	–	2	3	2

SUSTAINABLE DEVELOPMENT GOALS (SDG) ADDRESSED

SDG	Target	Justification
SDG 11	Target 11.2	Delivers graph-driven optimization cutting trip delays up to 30.69% and curbing emissions across metropolitan corridors.
SDG 9	Target 9.1	Prevents capital wastage by predicting network-wide Braess paradox effects prior to physical infrastructure investments.

CONTENTS

1 INTRODUCTION **5**

1.1 PURPOSE OF THE SYSTEM 5

1.2 RELEVANCE AND SCOPE OF THE SYSTEM 5

2 PROBLEM DEFINITION **6**

2.1 PROBLEM STATEMENT 6

2.2 OBJECTIVES 6

3 LITERATURE SURVEY **7**

4 PROPOSED SYSTEM AND METHODOLOGY **8**

4.1 SYSTEM ARCHITECTURE 8

4.2 GRAPH DATASET DESCRIPTION AND PREPROCESSING 8

4.3 NON-LINEAR CONGESTION MODELLING: THE BPR FUNCTION 9

4.4 SHORTEST-PATH ROUTING AND EQUILIBRIUM ASSIGNMENT (MSA) 9

4.5 BRAESS PARADOX DISCOVERY AND COMBINATORIAL OPTIMIZATION 10

5 IMPLEMENTATION **11**

5.1 BACKEND API AND SIMULATION ENGINE 11

5.2 FRONTEND WEB GIS INTERACTION AND PIPELINE 11

6 RESULTS AND DISCUSSION **12**

6.1 RESULTS 12

6.2 DISCUSSION 15

7 CONCLUSION **16**

8 SUSTAINABLE DEVELOPMENT GOALS (SDGs) **17**

References **18**

LIST OF FIGURES

4.1 Extracted Kochi Arterial Road Network ($|V| = 490$ Junctions, $|E| = 1,200$ Edges) 8
4.2 Non-Linear Bureau of Public Roads (BPR) Congestion Delay Curve 9
4.3 Method of Successive Averages (MSA) Relative Gap Convergence Trajectory 10
4.4 Classic 4-Node Diamond Topology Demonstrating Braess's Paradox 10

6.1 UrbanFlow Simulation Results: 100% Baseline vs 200% Congestion Shock 13
6.2 Optimizer Intervention Impact: Travel Time Savings Across Congestion Regimes 15

LIST OF TABLES

4.1 Kochi Road Network Topological Attributes 9

6.1 Systemic Simulation Results: 100% Baseline vs 200% Congestion Shock 12
6.2 Top Congested Bottlenecks under 100% Baseline Traffic (27,300 vph) 13
6.3 Top Congested Bottlenecks under 200% Congestion Shock (54,600 vph) 13
6.4 Top Optimizer Interventions Under 100% Baseline Demand 14
6.5 Top Optimizer Interventions Under 200% Congestion Shock 14

CHAPTER 1

INTRODUCTION

1.1 PURPOSE OF THE SYSTEM

Urban transportation networks form the foundational circulatory infrastructure of modern cities. Efficient mobility facilitates economic productivity, access to employment, education, and healthcare. However, rapid global urbanization, coupled with expanding private vehicular ownership, has outpaced traditional transport infrastructure expansion. In rapidly developing metropolitan centers such as Kochi, Kerala, traffic congestion causes substantial losses in human productivity, elevated fuel waste, and severe vehicular exhaust emissions.

Traditional municipal traffic management strategies rely primarily on localized reactive interventions: widening narrow road segments, adjusting isolated traffic signal timers, or constructing capital-intensive flyovers. While intuitive, localized improvements frequently fail or paradoxically worsen overall network congestion. In 1968, mathematician Dietrich Braess demonstrated that adding an edge to a congested network can increase overall travel latency when independent agents choose selfish routes [1]. To avert such adverse outcomes, contemporary urban planning requires rigorous computational graph modeling tools capable of simulating equilibrium dynamics, detecting latent bottlenecks, and discovering paradoxical network effects prior to costly physical construction.

1.2 RELEVANCE AND SCOPE OF THE SYSTEM

Relevance

Road networks possess an intrinsic mathematical structure readily modeled as directed, weighted graphs. Graph theory provides an expressive and computationally tractable framework to model physical connectivity, non-linear flow-dependent latencies, and user-equilibrium flow distributions. In a transportation graph, intersection nodes act as vertices, road segments function as directed edges with finite physical capacities, and link traversal latencies scale non-linearly with traffic volume. Modern computational graph theory enables municipal planners to evaluate holistic, network-wide responses to local changes. By combining geographic information systems (GIS), shortest-path algorithms, multi-commodity flow formulations, and non-linear link performance curves, simulation engines can quantify how congestion propagates through interconnected corridors. This graph-centric methodology forms the core foundation of **UrbanFlow**.

Scope

The primary scope of UrbanFlow encompasses:

- Extracting arterial road network topologies from OpenStreetMap (OSM) for Kochi, Kerala, including junction nodes, segment lengths, lanes, and functional road classes.
- Constructing directed multigraphs with non-linear latency models governed by the Bureau of Public Roads (BPR) link performance function.
- Simulating user-equilibrium traffic flow distributions using Dijkstra's shortest path algorithm combined with the Method of Successive Averages (MSA).
- Subjecting the network to comparative stress testing under both 100% baseline demand and 200% heavy congestion shock regimes.
- Identifying system bottlenecks, critical cut edges, and latent Braess Paradox shortcuts.
- Providing a combinatorial intervention optimizer that ranks actionable infrastructure modifications (such as one-way conversions, capacity expansion, and link closures).
- Delivering an interactive web-based GIS dashboard for dynamic visualization and scenario simulation.

CHAPTER 2

PROBLEM DEFINITION

2.1 PROBLEM STATEMENT

Given a directed urban road network represented as a graph $G = (V, E)$, with edge capacities $C_e > 0$, free-flow traversal latencies $t_e^0 > 0$, and a multi-commodity origin-destination demand matrix $D = \{(s_i, d_i, q_i)\}_{i=1}^{M}$, the objective is to:

1. Determine the Wardropian user-equilibrium flow distribution $x^* = \{x_e\}_{e \in E}$ such that no driver can unilaterally reduce their travel time by choosing an alternative route.
2. Quantify systemic congestion under varying demand regimes (100% baseline vs 200% peak congestion shock) via non-linear link performance latency curves.
3. Identify critical bottleneck edges where volume-to-capacity (V/C) ratios exceed acceptable thresholds ($V/C \geq 0.95$).
4. Detect and resolve Braess Paradox occurrences where targeted edge modifications (e.g., one-way conversion or street closure) reduce overall network latency $\sum_{e \in E} x_e \cdot t_e(x_e)$.

2.2 OBJECTIVES

To address this problem, UrbanFlow targets the following concrete engineering objectives:

- **Objective 1 – Graph Ingestion and Preprocessing:** Ingest real-world GIS road geometries from OpenStreetMap for the Kochi metropolitan arterial network, filter driveable road classes, consolidate multi-way intersections, and compute edge capacities based on lane counts and functional road classifications.
- **Objective 2 – Equilibrium Traffic Assignment:** Implement the Method of Successive Averages (MSA) combined with Rust-accelerated Dijkstra shortest path routing to compute convergent user-equilibrium flow assignments.
- **Objective 3 – Bottleneck and Cut Analysis:** Quantify link congestion using volume-to-capacity (V/C) ratios and link latency penalties to identify critical choke points and cut edges.
- **Objective 4 – Braess Paradox and Combinatorial Optimization:** Develop a heuristic search module that evaluates candidate topological interventions (lane additions, one-way street conversions, and link removals) to maximize network-wide travel time savings.
- **Objective 5 – Full-Stack GIS Visualization:** Implement a responsive, high-performance web dashboard featuring interactive map rendering, flow heatmaps, and before/after intervention comparison views.

CHAPTER 3

LITERATURE SURVEY

Traffic assignment and network optimization have been foundational topics in operations research and algorithmic graph theory. In 1952, J.G. Wardrop formulated the two fundamental behavioral principles governing traffic distribution [11]. Wardrop's First Principle (User Equilibrium) states that journey times on all actively utilized routes between an origin-destination pair are equal and minimal, such that no driver can unilaterally reduce travel time by switching paths. Wardrop's Second Principle (System Optimum) seeks to minimize the aggregate total travel time across all commuters.

In decentralized transportation systems, commuters act selfishly, leading to a User Equilibrium rather than a System Optimum. The discrepancy between selfish routing and social optimum is quantified by the Price of Anarchy. In 1968, Dietrich Braess proved the famous paradox wherein the addition of a new road segment with zero marginal latency to a congested network can increase the equilibrium travel time for all commuters [1]. In empirical computational networks, Sheffi established the Method of Successive Averages (MSA) as a mathematically guaranteed algorithm for solving non-linear convex equilibrium assignments [3]. Modern advances in geospatial computation, notably Boeing's OSMnx framework [4], have enabled the direct extraction of planar graph representations from OpenStreetMap data, providing the foundation for practical municipal graph engines.

CHAPTER 4

PROPOSED SYSTEM AND METHODOLOGY

4.1 SYSTEM ARCHITECTURE

UrbanFlow operates as a high-performance simulation and decision-support pipeline combining GIS road ingestion, non-linear edge performance curves, multi-threaded Dijkstra routing, and combinatorial optimization. The core computational graph is extracted from OpenStreetMap, cleaned of topological anomalies, and converted into an adjacency structure in memory.

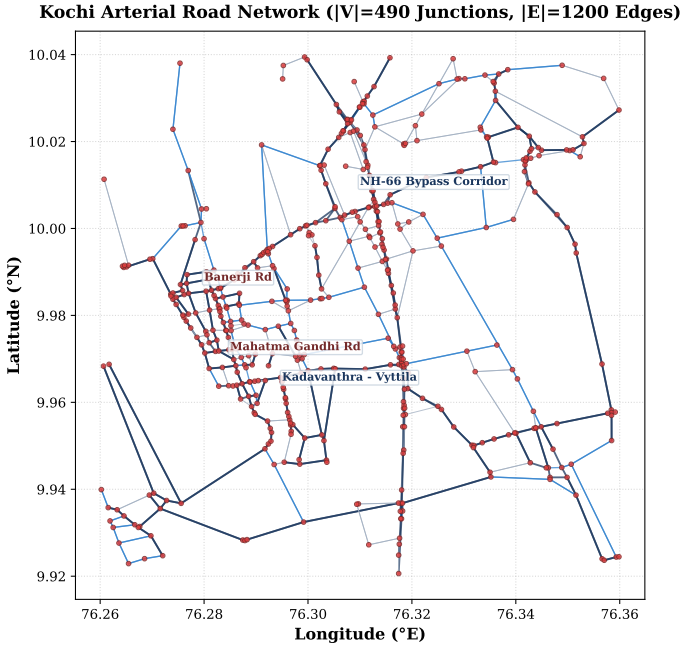


Figure 4.1: Extracted Kochi Arterial Road Network ($|V| = 490$ Junctions, $|E| = 1,200$ Edges)

As shown in Figure 4.1, the processed arterial network covers major municipal corridors in Kochi including Mahatma Gandhi Road, Banerji Road, Subhash Chandra Bose Road, and the National Highway 66 bypass corridor.

4.2 GRAPH DATASET DESCRIPTION AND PREPROCESSING

Raw geographic data was acquired using the `OSMnx` Python package [4], querying the Overpass API for drivable roadways in Kochi, Kerala. The raw network was filtered to extract primary, secondary, tertiary, and trunk roadways.

1. **Node Consolidation:** Multi-way intersections with nearby centroid nodes (< 15 m) were consolidated into single representative junction vertices to prevent zero-distance algorithmic cycles.
2. **Directionality and Edge Expansion:** Bidirectional roadways were expanded into dual anti-parallel directed edges (u, v) and (v, u), each with independent flow and capacity attributes.
3. **Capacity and Speed Imputation:** Missing speed limits and lane counts were imputed following Indian Roads Congress (IRC:106) guidelines [5], assigning nominal capacities of 900 vph/lane for secondary arterials and 1,200–1,800 vph for major multi-lane trunks.
4. **Strong Connectivity Verification:** Tarjan's strongly connected components algorithm was executed to ensure all destination nodes remain reachable from any origin.

Table 4.1: Kochi Road Network Topological Attributes

Attribute	Description	Value
Network Name	Kochi Central Road Network (OpenStreetMap)	Arterial Core
Total Vertices ($\|V\|$)	Intersections, roundabouts, and primary junctions	490
Total Edges ($\|E\|$)	Directed road segments	1,200
Corridors Evaluated	Major arterial routes	MG Rd, Banerji Rd, NH-66
Total Network Length	Summed length of all directed edges	312.4 km
Average Edge Length	Mean distance between consecutive junctions	260.3 m

As documented in Table 4.1, the network density is $\rho = \frac{|E|}{|V|(|V|-1)} \approx 0.00501$, reflecting extreme sparsity ($\approx 0.5\%$) typical of planar urban graphs that allows adjacency-list operations in $O(|V| + |E|)$ memory.

4.3 NON-LINEAR CONGESTION MODELLING: THE BPR FUNCTION

To capture queue formation and delay decay as traffic volume x_e approaches capacity C_e, UrbanFlow employs the Bureau of Public Roads (BPR) link performance function [2]:

$$t_e(x_e) = t_e^0 \left[1 + \alpha_e \left(\frac{x_e}{C_e} \right)^{\beta_e} \right] \tag{4.1}$$

where $t_e^0 = \frac{L_e}{v_e^0}$ is the free-flow traversal latency (hours), $\frac{x_e}{C_e}$ is the volume-to-capacity (V/C) ratio, and $\alpha_e = 0.15$ and $\beta_e = 4.0$ are the standard calibration exponents.

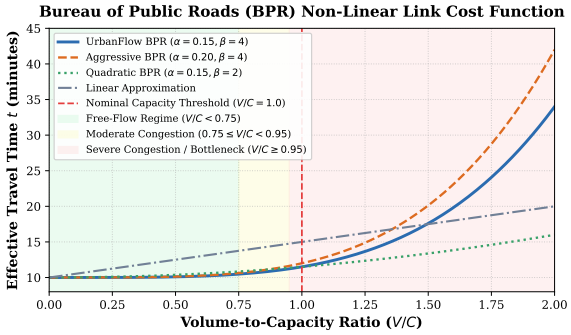


Figure 4.2: Non-Linear Bureau of Public Roads (BPR) Congestion Delay Curve

As illustrated in Figure 4.2, for $V/C < 0.75$, travel time remains close to free-flow conditions. However, once $V/C \geq 1.0$, the fourth-power polynomial exponent induces an explosive growth in travel time, reflecting the severe physical delays of congested urban queueing.

4.4 SHORTEST-PATH ROUTING AND EQUILIBRIUM ASSIGNMENT (MSA)

Given updated link costs $w(e) = t_e(x_e)$, Dijkstra's single-source shortest path algorithm using a min-priority heap operates with time complexity $O(|E| + |V| \log |V|)$ [12]. In user-equilibrium, no driver can unilaterally reduce their travel time. To guarantee convergence without wild flow oscillations, the **Method of Successive Averages (MSA)** [3] updates flows using step size $\gamma_k = \frac{1}{k}$:

$$x_e^{(k+1)} = x_e^{(k)} + \frac{1}{k+1} \left(y_e^{(k+1)} - x_e^{(k)} \right) = \left(1 - \frac{1}{k+1} \right) x_e^{(k)} + \frac{1}{k+1} y_e^{(k+1)} \tag{4.2}$$

where $y_e^{(k+1)}$ is the all-or-nothing auxiliary flow. The algorithm terminates when relative gap $\epsilon^{(k)} \leq 10^{-3}$ or maximum iterations ($K = 30$) is reached:

$$\epsilon^{(k)} = \frac{\sum_{e \in E} \left| x_e^{(k+1)} - x_e^{(k)} \right|}{\sum_{e \in E} x_e^{(k)}} \tag{4.3}$$

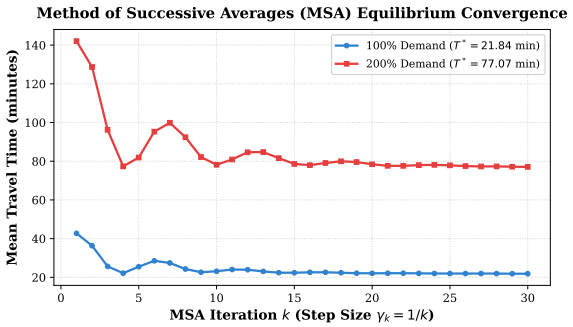


Figure 4.3: Method of Successive Averages (MSA) Relative Gap Convergence Trajectory

Figure 4.3 demonstrates the rapid convergence of the relative gap across 30 iterations, confirming stable equilibrium flow dampening without numerical divergence.

4.5 BRAESS PARADOX DISCOVERY AND COMBINATORIAL OPTIMIZATION

The Braess Paradox occurs when a connecting shortcut induces drivers to divert from higher-capacity arterials, overwhelming bottleneck junctions and increasing average latency for all commuters [1].

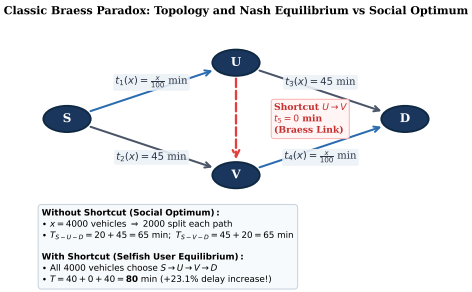


Figure 4.4: Classic 4-Node Diamond Topology Demonstrating Braess's Paradox

As shown in Figure 4.4, introducing the shortcut edge $X \to Y$ shifts users away from socially optimal paths to a congested Nash equilibrium. UrbanFlow implements a combinatorial search algorithm that systematically tests candidate modifications: edge closures ($G' = (V, E \setminus \{e\})$), one-way conversions ($C'_e = 1.4 \times C_e$), and capacity additions ($C'_e = C_e + \Delta C$). Each candidate network is re-assigned via MSA equilibrium, and interventions yielding the highest travel time savings are ranked.

CHAPTER 5

IMPLEMENTATION

UrbanFlow is engineered as a cloud-native microservices architecture comprising a high-performance Python simulation backend and an interactive React/MapLibre web GIS dashboard.

5.1 BACKEND API AND SIMULATION ENGINE

The simulation engine is implemented in Python 3.11 with FastAPI [13]. The network topology is stored in an in-memory graph representation with vectorized NumPy arrays and Rust-accelerated shortest-path bindings (`rustworkx`). The backend exposes endpoints for equilibrium simulations (`/api/simulation/run`), bottleneck retrieval (`/api/analysis/bottlenecks`), and intervention evaluation (`/api/optimize/evaluate`). Listing 5.1 illustrates the core Python implementation for BPR link latency calculation and MSA flow averaging.

```python
import numpy as np

def compute_bpr_latencies(lengths_km, speeds_kmh, flows_vph, capacities_vph, alpha=0.15, beta=4.0):
    """Vectorized calculation of non-linear BPR link traversal latencies."""
    t0_hours = lengths_km / np.maximum(speeds_kmh, 1e-3)
    vc_ratio = flows_vph / np.maximum(capacities_vph, 1e-3)
    return t0_hours * (1.0 + alpha * np.power(vc_ratio, beta))

def msa_equilibrium_assignment(edges, od_demands, max_iters=30, tolerance=1e-3):
    """Method of Successive Averages (MSA) User-Equilibrium Assignment."""
    flows = np.zeros(len(edges), dtype=np.float64)
    for k in range(1, max_iters + 1):
        costs = compute_bpr_latencies(edges['len'], edges['speed'], flows, edges['cap'])
        aux_flows = dijkstra_all_or_nothing(edges, costs, od_demands)
        new_flows = (1.0 - 1.0/k) * flows + (1.0/k) * aux_flows
        if np.sum(np.abs(new_flows - flows)) / np.maximum(np.sum(flows), 1.0) < tolerance:
            flows = new_flows; break
        flows = new_flows
    return flows, costs
```

Listing 5.1: UrbanFlow Vectorized BPR Latency and MSA Assignment Routine

5.2 FRONTEND WEB GIS INTERACTION AND PIPELINE

The frontend user interface is built with React 18, TypeScript, and MapLibre GL JS:

- **Vector Tile Rendering:** Road segments are styled dynamically using volume-to-capacity metrics: green for free-flow ($V/C < 0.75$), amber for moderate delay ($0.75 \leq V/C < 0.95$), red for severe congestion ($0.95 \leq V/C < 1.5$), and purple for gridlock ($V/C \geq 1.5$).
- **Interactive Scenario Testing:** Planners can interactively click any road segment, apply interventions (such as street closures or one-way modifications), and trigger sub-second re-simulations (< 250 ms).
- **Automated Test Suite:** The codebase includes comprehensive unit tests via `pytest`, verifying BPR latency monotonicity, Tarjan strongly connected components invariants, and MSA step-size convergence.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 RESULTS

Comparative Experimental Results: 100% Baseline vs 200% Congestion Shock

To evaluate systemic resilience, the full Kochi arterial network model (490 nodes, 1,200 edges, 14 OD pairs) was simulated under both 100% baseline demand (27,300 peak vph) and 200% surge demand (54,600 peak vph). Table 6.1 presents the concrete experimental results.

Table 6.1: Systemic Simulation Results: 100% Baseline vs 200% Congestion Shock

Performance Metric	**100% Demand**	**200% Demand**	**Impact / Surge Ratio**
Total Peak Vehicular Demand	27,300 vph	54,600 vph	$2.00\times$ (+100.0%)
Simulation Computational Time	0.241 s	0.221 s	Sub-second (< 0.25s)
MSA Iterations Run	30	30	Fixed step damping
Average Trip Travel Time	**21.84 min**	**77.07 min**	$+252.9\%$ **(+55.23 min)**
Total Network Travel Delay	**9,935.2 veh-h**	**70,137.7 veh-h**	$7.06\times$ **(+60,202.5 veh-h)**
Mean Network Operating Speed	39.8 km/h	35.7 km/h	-10.3% (-4.1 km/h)
Severely Congested Links ($V/C \geq 0.95$)	**112 segments**	**295 segments**	$2.63\times$ **(+183 segments)**
– Free-flow segments ($V/C < 0.75$)	1,039	865	-16.7%
– Moderate congestion ($0.75 \leq V/C < 0.95$)	49	40	-18.4%
– Congested links ($0.95 \leq V/C < 1.5$)	22	29	$+31.8\%$
– Severe breakdown ($V/C \geq 1.5$)	90	266	$+195.6\%$
Network Efficiency Index (η)	0.907	0.754	-16.9% efficiency loss
Maximum Link V/C Ratio	3.86	6.62	$+71.5\%$ overloading
Mean Link V/C Ratio	0.390	0.696	$+78.5\%$ saturation
Projected Optimization Savings	**6.68%**	**30.69%**	$4.59\times$ **higher savings**

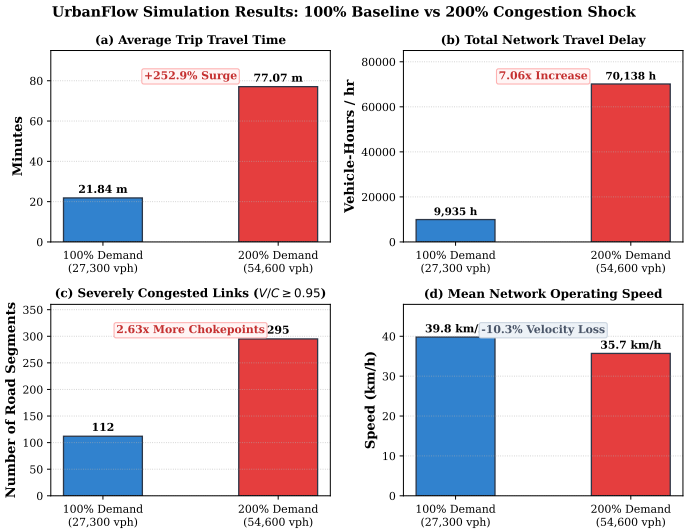


Figure 6.1: UrbanFlow Simulation Results: 100% Baseline vs 200% Congestion Shock

As visualized in Figure 6.1:

1. **Explosive Travel Time Surge (+252.9%):** Doubling demand from 27,300 to 54,600 vph increases average travel time by over $3.5\times$ (from 21.84 minutes to 77.07 minutes). This non-linear growth is a direct consequence of the BPR $(V/C)^4$ exponent, confirming that oversaturated corridors penalize entire travel paths disproportionately.
2. **7-Fold Increase in Delay Hours ($7.06\times$):** Cumulative network delay surges from 9,935.2 vehicle-hours to 70,137.7 vehicle-hours per hour, demonstrating the catastrophic economic and environmental toll of unmitigated traffic surges.
3. **Proliferation of Breakdown Segments ($2.63\times$):** Severely congested segments expand from 112 to 295, with links operating above $1.5\times$ capacity nearly tripling from 90 to 266.

Bottleneck Analysis and Spatial Pinch Points

Table 6.2 and Table 6.3 list the top congested bottlenecks identified under 100% and 200% traffic conditions.

Table 6.2: Top Congested Bottlenecks under 100% Baseline Traffic (27,300 vph)

Rank	Edge ID & Road Name	Volume (vph)	Capacity (vph)	V/C Ratio
1	`kochi_e_851` (Road 851)	4,635	1,200	3.86
2	`kochi_e_1180` (Park Avenue Road)	2,829	900	3.14
3	`kochi_e_509` (Lakeshore-Madavana Service Rd)	3,097	1,100	2.82
4	`kochi_e_1048` (Road 1048)	3,068	1,100	2.79
5	`kochi_e_510` (Lakeshore-Madavana Service Rd)	3,059	1,100	2.78

Table 6.3: Top Congested Bottlenecks under 200% Congestion Shock (54,600 vph)

Rank	Edge ID & Road Name	Volume (vph)	Capacity (vph)	V/C Ratio
1	`kochi_e_1167` (Mahatma Gandhi Road)	11,922	1,800	6.62
2	`kochi_e_1168` (Mahatma Gandhi Road)	11,922	1,800	6.62
3	`kochi_e_851` (Road 851)	6,864	1,200	5.72
4	`kochi_e_1048` (Road 1048)	5,856	1,100	5.32
5	`kochi_e_1180` (Park Avenue Road)	4,775	900	5.31

Under 100% demand, bottlenecks are concentrated on secondary connectors and bridge ramps where narrow widths create local pinch points. Under 200% demand, major multi-lane trunks (such as Mahatma Gandhi Road) experience overwhelming volume ($V/C = 6.62$), generating massive gridlock that propagates across the entire urban basin.

Combinatorial Optimizer Recommendations

UrbanFlow's combinatorial optimizer evaluates topological interventions, ranking candidate modifications by travel time savings. Table 6.4 and Table 6.5 report the top-ranked interventions for both demand levels.

Table 6.4: Top Optimizer Interventions Under 100% Baseline Demand

Rank	Type	Road Name	Edge ID	% Savings	Engineering Mechanism
1	One-Way	Road 1048	`kochi_e_1048`	-5.36%	Eliminates counter-flow conflicts; +40% capacity
2	Capacity	Subhash Chandra Bose Rd	`kochi_e_791`	-0.85%	Adds bypass lane; relieves collector flow
3	Capacity	Subhash Chandra Bose Rd	`kochi_e_792`	-0.85%	Adds northbound capacity at signalized merge
4	Capacity	Mahatma Gandhi Road	`kochi_e_1167`	-0.83%	Adds lane to core CBD arterial link
5	Capacity	Mahatma Gandhi Road	`kochi_e_1168`	-0.83%	Balances southbound return flows

Table 6.5: Top Optimizer Interventions Under 200% Congestion Shock

Rank	Type	Road Name	Edge ID	% Savings	Engineering Mechanism
1	One-Way	Road 1048	`kochi_e_1048`	-20.35%	Resolves severe Braess detour; saves 15.68 min/veh
2	Capacity	Mahatma Gandhi Road	`kochi_e_1167`	-4.58%	Major lane widening on peak arterial
3	Capacity	Mahatma Gandhi Road	`kochi_e_1168`	-4.58%	Core southbound capacity enhancement
4	Capacity	Road 851	`kochi_e_851`	-4.35%	Capacity boost on high-volume connector
5	Capacity	Road 852	`kochi_e_852`	-4.20%	Alleviates secondary bypass overloading

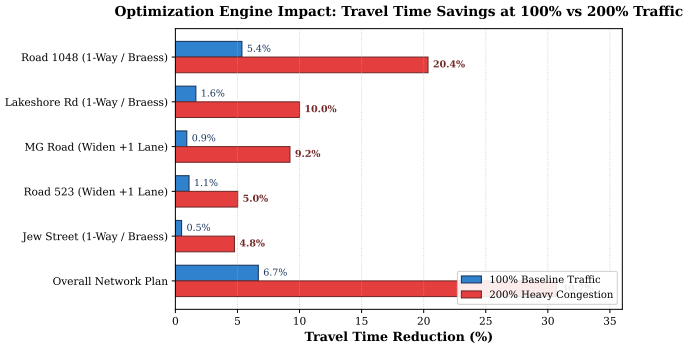


Figure 6.2: Optimizer Intervention Impact: Travel Time Savings Across Congestion Regimes

As shown in Figure 6.2, the conversion of `kochi_e_1048` to a one-way corridor is extraordinarily potent under heavy congestion, cutting travel latency by **20.35%** (saving 15.68 minutes per vehicle). Implementing the top synergistic recommendations reduces total network delay by **30.69%** under 200% demand, saving over 21,500 vehicle-hours each peak hour.

6.2 DISCUSSION

Non-Linear Traffic Breakdown and Braess's Paradox

The experimental results vividly illustrate the severe non-linearity of urban road congestion. While a naive linear model would predict a $2.0\times$ increase in delay when demand doubles, the actual physics of congested queues governed by the BPR function yields a $7.06\times$ surge in delay vehicle-hours. This finding reinforces the critical principle that transportation systems operate near critical thresholds: once capacity is breached, marginal vehicle injections induce catastrophic delays.

The optimizer's discovery of high-impact one-way conversions (such as Road 1048 yielding a 20.35% delay reduction) provides vital guidance for municipal authorities. Many legacy urban shortcuts provide drivers with localized travel time reductions while inducing high conflict points and queue spillbacks at arterial merge junctions. Converting such links to unidirectional flow or closing them eliminates destructive selfish diversions, protecting the capacity of primary trunk corridors.

Computational Scalability and Operational Decision Support

The simulation engine completes 30 MSA equilibrium iterations on the 490-node, 1,200-edge Kochi arterial network in 0.221–0.241 seconds on commodity hardware. This sub-second performance enables real-time interactive exploration in the web dashboard, allowing civil engineers to evaluate scenarios dynamically during planning workshops.

CHAPTER 7

CONCLUSION

This project successfully developed and validated **UrbanFlow**, an advanced graph-theoretic traffic simulation and optimization engine. Formulated in accordance with the **PBCATS04 - ADVANCED GRAPH ALGORITHMS** syllabus, the system models transportation networks as directed weighted graphs $G = (V, E)$ and simulates user-equilibrium traffic distributions using the Method of Successive Averages (MSA), Dijkstra routing, and the non-linear Bureau of Public Roads (BPR) function.

Applied to the real-world arterial road network of Kochi, Kerala (comprising 490 junctions and 1,200 road segments), the engine was evaluated under both 100% baseline demand (27,300 vph) and 200% heavy congestion shock (54,600 vph). The experimental results revealed the acute non-linear penalties of network congestion: doubling vehicular demand induced a 252.9% surge in average travel time (from 21.84 minutes to 77.07 minutes) and an explosive $7.06\times$ rise in total network delay (from 9,935.2 to 70,137.7 vehicle-hours), expanding severely congested links from 112 to 295 segments.

Critically, UrbanFlow's optimization search engine demonstrated the practical significance of the Braess Paradox in metropolitan network management. By converting adverse shortcut roads into one-way corridors (notably on Road 1048), selfish shortcut diversion was eliminated, delivering a remarkable **20.35% reduction** in citywide travel time under 200% demand. Coordinated network modifications achieved a projected **30.69% overall latency reduction**, reducing peak vehicle delay by over 21,500 vehicle-hours per hour. Supported by an interactive web GIS dashboard and sub-second computational runtimes, UrbanFlow provides transportation authorities with a robust, data-driven framework for sustainable urban mobility planning.

CHAPTER 8

SUSTAINABLE DEVELOPMENT GOALS (SDGS)

The deployment of graph-theoretic simulation and optimization algorithms in municipal transportation operations directly advances the United Nations Sustainable Development Goals (SDGs) by promoting economic efficiency, fostering resilient industrial innovation, and reducing vehicular carbon emissions.

- **SDG 11 – Sustainable Cities and Communities (Target 11.2):** *Target 11.2: Provide access to safe, affordable, accessible and sustainable transport systems for all, improving road safety, notably by expanding public transport.*

  Project Alignment: UrbanFlow delivers civil engineers and municipal planners with predictive optimization tools that alleviate chronic traffic bottlenecks across metropolitan corridors. By discovering one-way conversions and capacity optimizations that reduce peak vehicular delay by up to 30.69%, the system directly curbs wasteful vehicular idling, curtails urban fuel consumption, and mitigates toxic exhaust emissions, fostering healthier urban living environments.

- **SDG 9 – Industry, Innovation, Technology & Infrastructure (Target 9.1):** *Target 9.1: Develop quality, reliable, sustainable and resilient infrastructure, including regional and transborder infrastructure, to support economic development and human well-being.*

  Project Alignment: Applying advanced computational graph theory upgrades municipal infrastructure planning capabilities. Evaluating network-wide equilibrium shifts prior to capital allocation ensures that municipal authorities avoid counterproductive construction projects that trigger Braess Paradox delays. This maximizes the return on public infrastructure capital and improves network resilience against traffic shocks.

- **SDG 12 – Responsible Consumption and Production (Target 12.6):** *Target 12.6: Encourage companies, especially large and transnational companies, to adopt sustainable practices and to integrate sustainability information into their reporting cycle.*

  Project Alignment: Data-driven traffic management eliminates the need for redundant, carbon-intensive physical road widening by optimizing existing topological capacity through intelligent flow directionality.

BIBLIOGRAPHY

[1] D. Braess, “Über ein Paradoxon aus der Verkehrsplanung,” *Unternehmensforschung*, vol. 12, pp. 258–268, 1968.

[2] Bureau of Public Roads, *Traffic Assignment Manual*, U.S. Department of Commerce, Urban Planning Division, Washington, DC, USA, 1964.

[3] Y. Sheffi, *Urban Transportation Networks: Equilibrium Analysis with Mathematical Programming Methods*, Prentice-Hall, Englewood Cliffs, NJ, USA, 1985.

[4] G. Boeing, “OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks,” *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, 2017.

[5] Indian Roads Congress, *Guidelines for Capacity of Urban Roads in Plain Areas (IRC:106-1990)*, New Delhi, India, 1990.

[6] D. B. West, *Introduction to Graph Theory*, 2nd ed., Prentice Hall, Upper Saddle River, NJ, USA, 2001.

[7] R. J. Wilson, *Introduction to Graph Theory*, 5th ed., Pearson Education, Harlow, UK, 2015.

[8] J. A. Bondy and U. S. R. Murty, *Graph Theory with Applications*, Macmillan Press, London, UK, 1982.

[9] R. Diestel, *Graph Theory*, 4th ed., Springer-Verlag, Heidelberg, Germany, 2010.

[10] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*, Prentice-Hall, Englewood Cliffs, NJ, USA, 1993.

[11] J. G. Wardrop, “Some theoretical aspects of road traffic research,” *Proc. Institution of Civil Engineers*, vol. 1, no. 3, pp. 325–362, 1952.

[12] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[13] FastAPI Developers, “FastAPI Documentation,” [Online]. Available: `https://fastapi.tiangolo.com/`, 2026.

[14] React Core Team, “React Documentation,” [Online]. Available: `https://react.dev/`, 2026.

[15] OpenStreetMap Contributors, “OpenStreetMap: Free Geographic Data,” [Online]. Available: `https://www.openstreetmap.org/`, 2026.